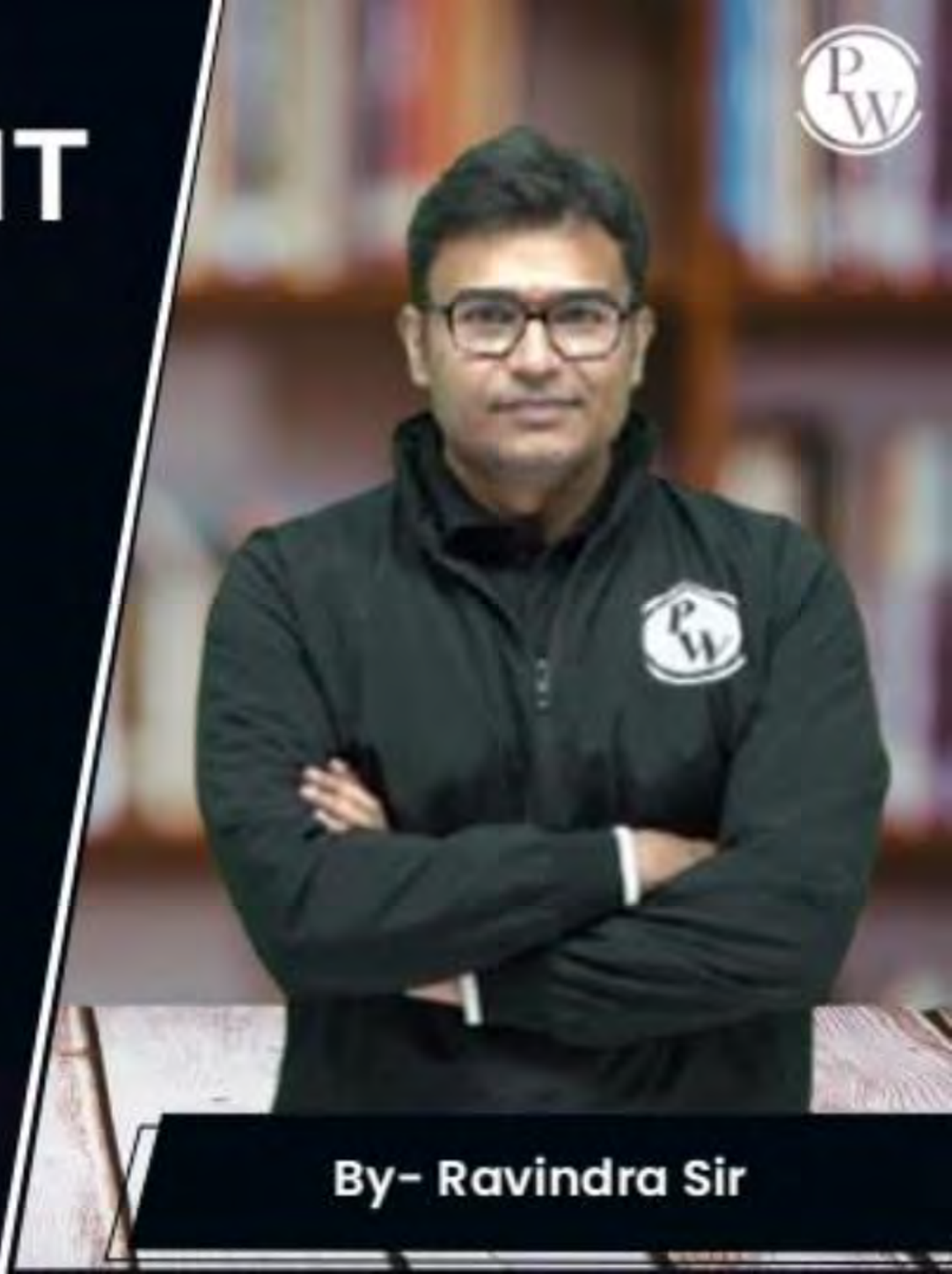
Computer Science & IT

COMPILER DESIGN
(CD)

Parsers

Lecture No. 1



By- Ravindra Sir

Recap of Previous Lecture



- Topic
- Topic
- Topic

Compiler

Topics to be Covered



- Topic
- Topic
- Topic

Deep into Subject



Unstoppable Indians: Stories to Ignite Student Motivation

Sabeer Bhatia – The Email Innovator

An IIT Kharagpur graduate, Sabeer Bhatia teamed up with Jack Smith to launch Hotmail over a single weekend in 1996, creating the first free web-based email service. Facing investor skepticism and technical hurdles, they iterated rapidly, growing to millions of users before selling to Microsoft for $400 million. His story demonstrates that bold prototypes and a bias for action can outpace convention—a powerful lesson for GATE aspirants to build, test, and learn quickly.



Unstoppable Indians: Stories to Ignite Student Motivation

Jagadish Chandra Bose – Wireless Pioneer & Plant Scientist Jagadish Chandra Bose (1858–1937) was a trailblazing scientist who, in 1895, built his own coherer and transmitter in Calcutta to send electromagnetic waves, predating Marconi's wireless experiments. He then invented the crescograph, a delicate instrument made from clockwork, glass, and metal that measured and amplified plant growth movements by micrometers. Bose demonstrated that plants respond to stimuli, published in English and Bengali, and inspired generations by keeping scientific knowledge free.



Unstoppable Indians: Stories to Ignite Student Motivation

Sam Pitroda – Architect of India's Digital Communications

After earning his PhD in electrical engineering, Sam Pitroda returned in 1975 to head the Centre for Development of Telematics (C-DOT). He designed India's first indigenous, low-cost digital switching systems—replacing unreliable imports—and portable satellite terminals that brought telephony to remote villages. Under his leadership, over 200 rural call offices connected millions of Indians for the first time. Later, as technology advisor to PM Rajiv Gandhi, he launched policies that democratized computer and internet access across government and academia.



Unstoppable Indians: Stories to Ignite Student Motivation

Raj Reddy – The AI Trailblazer

Raj Reddy left IIT Madras to pursue a PhD at Stanford, navigating cultural shifts and an emerging research frontier. He became one of the first professors of Artificial Intelligence, publishing pioneering work in speech recognition and robotics. His patience in exploring uncharted territories and commitment to academic rigor laid the foundation for modern AI. GATE students can learn from his example that curiosity-driven research and resilience are vital to mastering advanced topics.

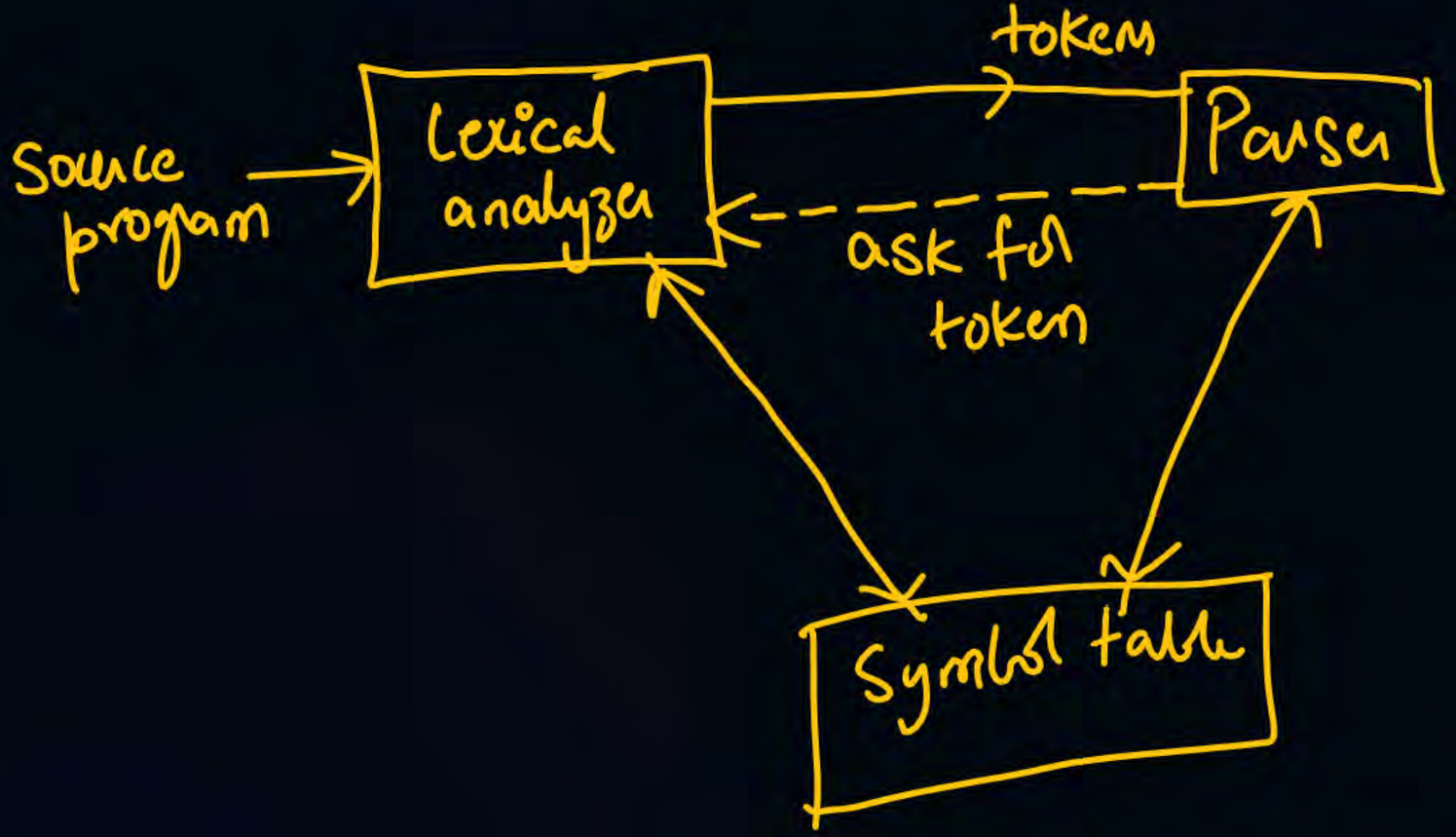

CD
LA
SA
Sem A
ICG
CO
TCG

Lexical analysis – (2 of 3)

Lexical analysis is the act of breaking source text into a set of words called tokens.

Each token is found by matching sequential characters to patterns.

All tokens are defined with regular grammar, and lexical analysers identifies strings of tokens and sends them to syntax analyzer for parsing


token
Source program
Lexical analyzer
Parser
ask for token
Symbol table

Gate :- 2004 :-

```
if (x>1){
    x=x+1;
} else {
    x=x-1;
}
```

→ Scanner / Lexical analizer (Finite automata)

lexical specification (Reg expressions)

IF, LP, ID, Relop, Num, RP, LB

Id, assgn, Id, Addop, Num, SC

RB, ELSE, LB

Id, assign, ID, Subop, Num, SC

RB

Gate 2015 1-

How many token?

```
int max ((x, y))
    int x, y;
 /* find max of x and y */
{
Return (x >= y ? x : y ));
}
```

25

Gate 2006 ⊢

| Printf | (| "%d Hai" | , | & | x |) | ; |

"%d Hai" — 1 token

8

Secondary tasks of lexical analyzer

1) Filtering comment lines and white spaces

White space characters are space, tab, newline are not required

Comments are not required for compilers

2) Associating the error messages with line number

A lexical analyzer reads the source code character by character,

while reading it, it remembers line number.

Ex: ; is missing at (line 5)

Tokens, pattern, lexeme :-

Token is a group of characters with logical meaning

Ex: id, keyword, Num etc

Pattern :- It is a rule that describes the characters that can be grouped into tokens. It is expressed as a Regular expression

Ex: id — $[A\text{-}Z\ a\text{-}z]\ [A\text{-}Z\ a\text{-}z]^*$

Lexeme :- It is the actual text / character stream that matches with the pattern and recognised as a token.

Ex :- float Key = 1.2 ;

Lexeme	Token	Pattern
float	float	float ✓
Key	id	[A-Z a-z] [A-Z a-z]*
= ✓	assgn	= ✓
1.2	num	any numeric constant.
;	SC ✓	; ✓

Errors that are detected in CA :-

1) Numerical literals are too long.

int a = 25262813754 5

2) long identifiers :-

C - 32 characters

int ababababba ---- a5 X

3) Ill formed numeric literal

int a = 1.b → LE

4) I/P characters that are not in the source langue

int a = #

How to handle errors in LA:-

There are four approaches to recover from lexical error

→ Delete: If there is a unknown character, then delete

→ Insert: Insert missing character

→ Transpose: We can transpose two characters

whiel → while

→ Replace: Replaces a character by another character

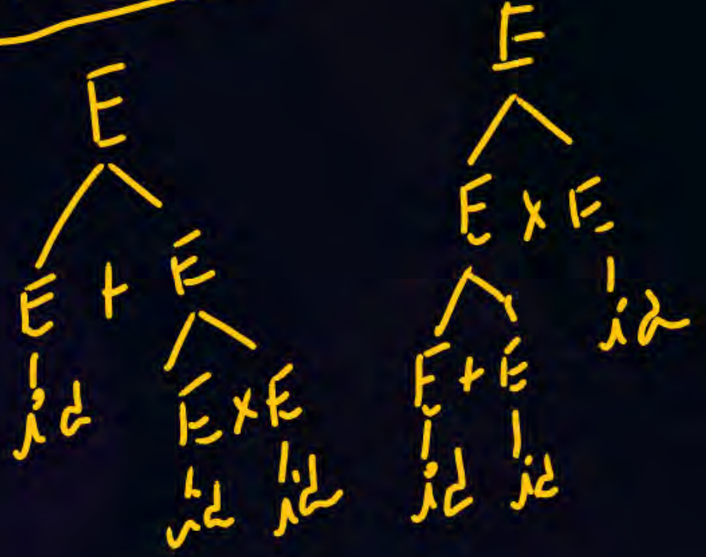
chrr → char

Grammar :-

$E \rightarrow E + E$
$\quad / E * E$
$\quad / id$

ambi

$id + id * id$



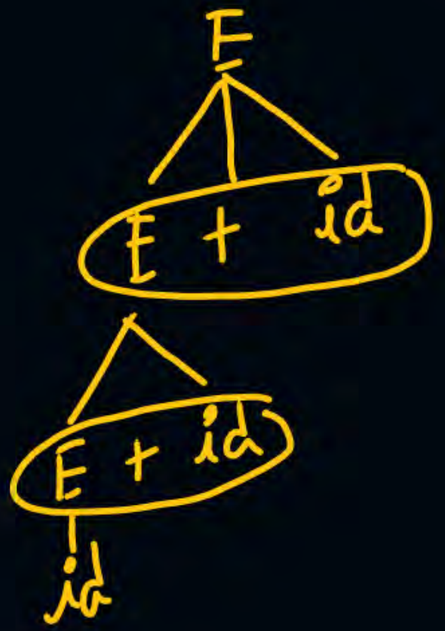
$E \to E+E \mid E*E \mid id$ } disambiguating $\longrightarrow$ $E \to E+id \mid id$

$id + id + id$

$id + id + id$

① 

② 



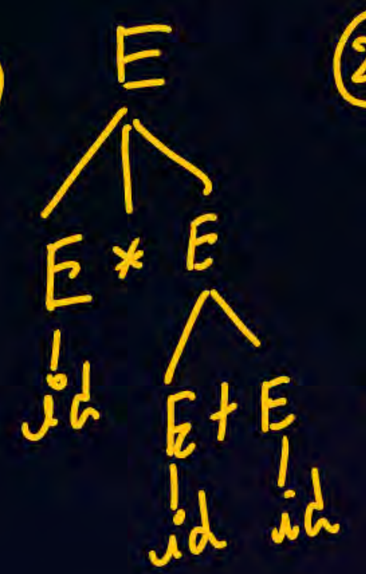
$(id + id) + id$

associativity is not taken

$E \rightarrow E+E$
$\quad / E*E$
$\quad / id$

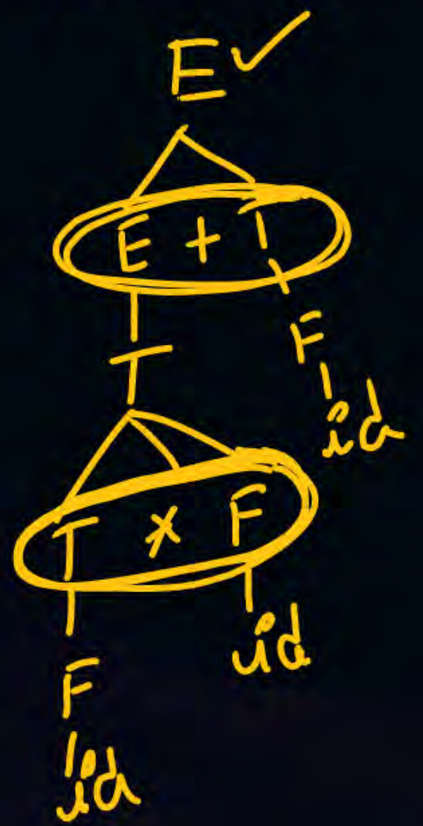
$E \rightarrow E+T / T$
$T \rightarrow T*F / F$
$F \rightarrow id$

$(id * id) + id$

① 

② 

$id * id + id$



$E \rightarrow E+E$
$\quad / E*E$
$\quad / id$

$id + id + id$

LMD

$E \Rightarrow \underline{E}+E$
$\Rightarrow id + E$
$\Rightarrow id + \underline{E}+E$
$\Rightarrow id + id + E$
$\Rightarrow id + id + id$

LMD

$E \Rightarrow \underline{E}+E$
$E \Rightarrow \underline{E}+E+E$
$E \Rightarrow id + \underline{E}+E$
$E \Rightarrow id + id + E$
$E \Rightarrow id + id + id$

$E \to E+E$
$\quad / E*E$
$\quad / id$

} ambiguous

$id + id + id$

'+' → left associative

(+) (+)

(+)(+)(+)

RMD

$E \to E+E$
$\Rightarrow E+id$
$\Rightarrow E+E+id$
$\Rightarrow E+E+id$
$\Rightarrow id+id+id$

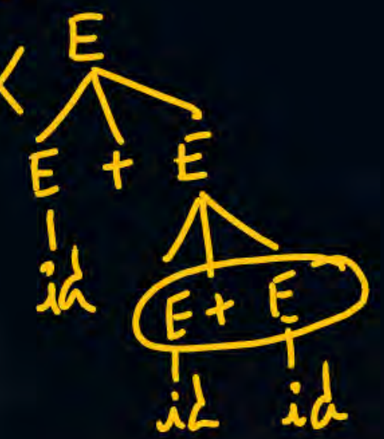
RMD

$E \Rightarrow E+E$
$\Rightarrow E+E+E$
$\Rightarrow E+E+id$
$\Rightarrow E+id+id$
$\Rightarrow id+id+id$

✓



X



$E \rightarrow E + E / id$

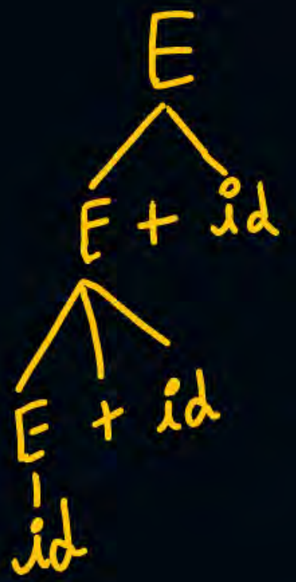
$id + id + id$





$E \rightarrow E + id / id$

$id + id + id$

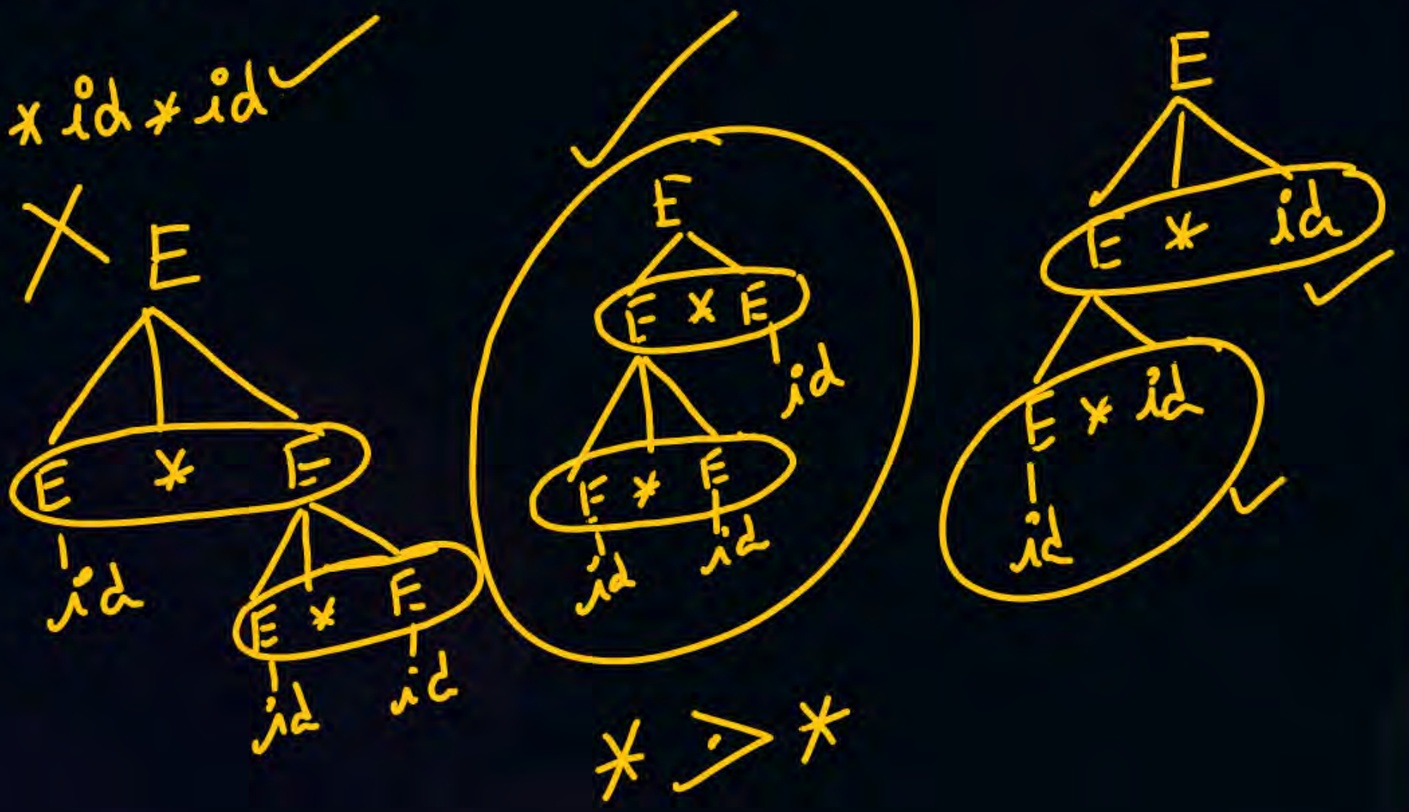


Note if a symbol is left recursive, then it will become left associative

$E \to E * E / id$ ✓

$E \to E * \underline{id} / id$ ✓

$id * id * id$ ✓



Left Recn – Left Ass

Righ Recn – Right Assn

↑

$2^{(3\uparrow 4)}$

$E \to E \uparrow E / id$

$id \uparrow id \uparrow id$

$(2 \uparrow (3 \uparrow 4))$

$E \to id \uparrow E / id$

$id \uparrow id \uparrow id$

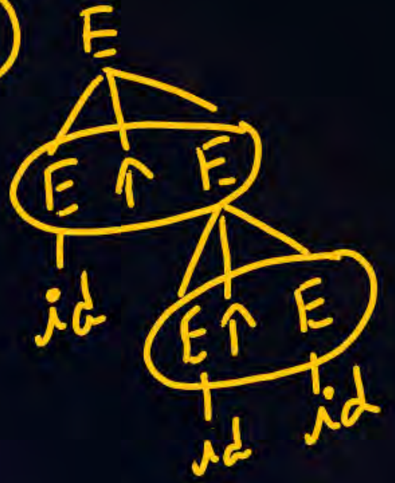
LR — LA

RR — RA

①



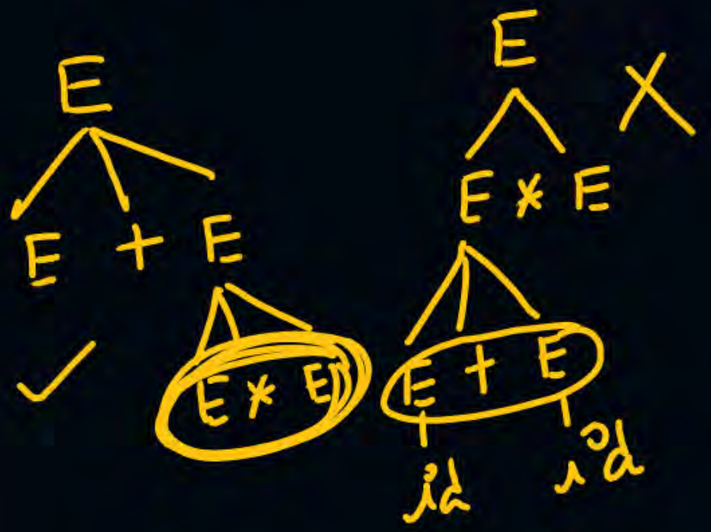
②





$E \rightarrow E + E$
$\quad / E * E$
$\quad / E \uparrow E$
$\quad / id$

$id + id * id$



precedence is not taken care

$* > +$

$E \rightarrow E + T \;/\; T$

$T \rightarrow T * F \;/\; F$ → unambiguous

$F \rightarrow id$

$* > +$

$id * id + id$



LR — LA

RR — RA

Levels – I +
II *
III ↑

$\uparrow > * > +$

$\pm, *, \uparrow$

$E \rightarrow E \pm T / T$

$T \rightarrow T * F / F$

$F \rightarrow G \uparrow F / G$

$G \rightarrow id$

$+ \gtrdot +$

$* \gtrdot *$

$\uparrow \lessdot \uparrow$

$\uparrow \gtrdot * \gtrdot +$

THANK - YOU